How Should Verifier Feedback Be Represented? A Controlled Study of Format and Specificity Effects on LLM Self-Correction

Anonymous Author(s)

Abstract

When an LLM corrects its own output using verifier feedback, does the *representation* of that feedback—its surface format and specificity level—matter? We decouple feedback representation from feedback content by fixing two deterministic verifiers (geometric property checking for SVG, static analysis for Python security) and systematically varying how their output is rendered. In a pre-specified factorial design spanning two 7–8B models (Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct) and two domains, Phase A ablates specificity (precise vs. generic) and Phase B further ablates format (natural language, raw JSON, hybrid). Our pre-specified headline claim—that precise feedback improves diagnostic reduction rate (DRR) by $\geq$10 percentage points universally—**fails**: mean $\Delta = -1.4$ pp across cells. Instead, specificity is domain-dependent: precise feedback yields $+12.2$ pp on Python for Qwen (p=0.021) but -19.9 pp on SVG (p<0.001). Format actively modulates specificity rather than serving as a neutral carrier: hybrid + precise achieves the best Qwen DRR (75.5%, p=0.002), while raw_json + generic achieves the best Llama DRR (79.9%). No single feedback configuration dominates across models and domains, arguing that self-correction pipelines should adapt feedback representation to the model–domain context rather than relying on a fixed template.

1 Introduction

LLM self-correction—prompting a model to revise its output given external feedback—has emerged as a practical strategy for improving code generation, where deterministic verifiers (compilers, test suites, static analyzers) provide reliable error signals Chen et al. [2024], Shinn et al. [2023], Madaan et al. [2023]. A growing body of evidence shows that external feedback is essential: without it, models frequently flip correct answers to incorrect ones, yielding net-negative correction Huang et al. [2024], Kamoi et al. [2024]. Given that feedback quality bottlenecks self-repair outcomes Olausson et al. [2024], a natural question arises: holding the verifier fixed, how should its output be *communicated* to the model?

Prior work has largely treated feedback representation as a fixed consequence of the tool. Compiler error messages, test-case traces, and linter reports arrive in tool-determined formats, and self-correction systems pass them through verbatim Chen et al. [2024], Yang et al. [2024b,c]. When studies do vary feedback, they typically change the *source*—comparing execution feedback against self-generated explanations Chen et al. [2024], or test failures against static-analysis reports Dolcetti et al. [2024]—simultaneously changing both what is detected and how it is expressed. This conflation makes it impossible to attribute gains to representation alone.

Meanwhile, a separate line of work demonstrates that prompt format significantly affects model behavior. Performance on identical tasks can swing by up to 40 percentage points across plain-text, Markdown, JSON, and YAML renderings He et al. [2024], and optimal format is task- and content-dependent Liu et al. [2025]. Chain-of-thought prompting Wei et al. [2022] and structured

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

decomposition Zhou et al. [2023] further illustrate that *how* information is organized in a prompt can matter as much as *what* information is included. If prompt formatting affects task performance so dramatically, feedback formatting should affect self-correction too—yet no controlled study has tested this.

We pose a direct research question: **how should verifier feedback be represented to maximize LLM self-correction?** Specifically, we decompose feedback representation into two orthogonal dimensions—*specificity* (how much diagnostic detail is exposed) and *format* (the surface structure: natural language, raw JSON, or a hybrid of both)—and measure their effects on correction quality while holding the underlying verifier constant.

Our experimental design uses two deterministic verifiers: an SVG geometric property checker Cao et al. [2025] that evaluates spatial relationships (coverage, overlap, containment, proportion) and a Python static analyzer combining Bandit PyCQA [2024a] security scanning with Pylint PyCQA [2024b] code quality checks. Both emit diagnostics through a unified schema that cleanly separates detection from presentation: the same diagnostic can be rendered as a terse severity-only message (generic) or a detailed report with line numbers, metric values, and fix suggestions (precise), in any of three surface formats. We test two 7–8B instruction-tuned models—Qwen2.5-7B-Instruct Yang et al. [2024a] and Llama-3.1-8B-Instruct Grattafiori et al. [2024]—under greedy decoding in a single-turn correction protocol. Phase A fixes format at natural language and ablates specificity across both domains ($2 \times 2 \times 2 = 8$ cells). Phase B, triggered by Phase A findings, further ablates three formats on the Python domain where specificity showed the clearest signal ($2 \times 3 \times 2 = 12$ cells).

The pre-specified headline claim—that precise feedback improves the macro diagnostic reduction rate (DRR) by $\geq$10 pp across all model–domain cells—fails decisively, with mean $\Delta = -1.4$ pp. The failure, however, reveals a structured pattern rather than noise. Specificity is *domain-dependent*: precise feedback yields a significantly positive effect on Python but a significantly negative effect on SVG for Qwen, a crossover interaction driven by the mismatch between detailed geometric diagnostics and weak spatial reasoning (Table 1). Phase B shows that format is not a neutral carrier: it actively modulates the specificity effect. Qwen's best configuration is hybrid + precise (75.5% DRR, p=0.002, surviving Bonferroni correction), while Llama performs best under raw_json + generic (79.9% DRR)—opposite corners of the design space (Table 3). These model-dependent optima rule out any universal "best format" and demonstrate that feedback representation is a first-class design variable in self-correction pipelines.

This work makes four contributions:

1. **Controlled factorial design.** The first study to disentangle feedback specificity and format as independent variables in LLM self-correction, using deterministic verifiers that fix feedback content while varying only its representation (§3).
2. **Domain-dependent specificity.** Evidence that the effect of feedback detail reverses between domains: precise diagnostics help rule-based tasks (Python security, $+12.2$ pp for Qwen) but hurt structural tasks (SVG geometry, -19.9 pp), with opposing directions of significance suggesting a domain$\times$specificity interaction for Qwen (§4.1).
3. **Format as active modulator.** The discovery that surface format amplifies or suppresses the specificity effect in a model-dependent manner, with hybrid + precise optimal for Qwen and raw_json + generic optimal for Llama (§4.2).
4. **Transparent hypothesis testing.** A pre-specified decision framework with honest reporting of headline hypothesis failure, demonstrating that the absence of a universal specificity advantage is itself an informative finding (§4.3).

2 Related Work

2.1 LLM Self-Correction and Self-Refinement

A growing body of work explores whether LLMs can improve their own outputs through iterative revision. Self-Refine Madaan et al. [2023] prompts a single LLM to generate feedback on its output and refine accordingly, reporting gains across seven diverse tasks. Reflexion Shinn et al. [2023] converts binary environment signals into verbal self-reflections stored in episodic memory, improving performance on decision-making and coding benchmarks. Self-Debug Chen et al. [2024] applies this

paradigm to code, comparing three feedback types—simple feedback, unit-test traces, and "rubber duck" explanations—that implicitly vary in specificity. Notably, Self-Debug simultaneously changes both the feedback *source* (execution vs. self-explanation) and its specificity, making it difficult to attribute gains to representation alone; our design fixes the underlying verifier and varies only the rendering, isolating the representation effect.

The reliability of self-generated feedback has, however, been challenged. Huang et al. [2024] demonstrate that without external oracles, LLMs frequently flip correct answers to incorrect ones, yielding net-negative correction on reasoning tasks. A critical survey by Kamoi et al. [2024] reaches a similar conclusion: successful self-correction in prior work almost always relies on external feedback or task-specific fine-tuning. Olausson et al. [2024] further show that self-repair in code generation is bottlenecked by feedback quality—using a stronger model as the feedback source yields substantially larger gains.

These findings motivate our setup: rather than asking whether LLMs *can* self-correct, we take external verifier feedback as given and ask how its *representation* affects correction outcomes.

2.2 Verifier and Execution Feedback for Code

External feedback sources—compilers, test suites, static analyzers—have been widely used to guide LLM code generation. CodeT Chen et al. [2023] and AlphaCode Li et al. [2022] use test execution to filter or rank candidate solutions. LEVER Ni et al. [2023] trains a learned verifier to rerank generations. LDB Zhong and Wang [2024] segments programs into basic blocks and verifies intermediate variable values against specifications, analogous to step-through debugging. MGDebugger Shi et al. [2024] decomposes code into hierarchical sub-functions and debugs bottom-up. Dolcetti et al. [2024] show that open-source LLMs can substantially repair flawed code when provided with test failures or static-analysis vulnerability reports. At the training level, RLTF Liu et al. [2023] uses multi-granularity unit-test feedback as reward signals, and RLEF Gehring et al. [2025] trains models end-to-end with execution feedback via reinforcement learning.

SWE-agent Yang et al. [2024b] wraps an LLM in an agent loop with tool feedback (linter output, test results) for repository-level bug fixing, while InterCode Yang et al. [2024c] provides an interactive coding environment where execution feedback drives multi-turn repair. Both systems demonstrate the value of rich tool feedback but use fixed feedback formats determined by the underlying tools.

A shared characteristic of these approaches is their focus on the *presence* of feedback—comparing feedback-assisted generation against unassisted baselines. The content and format of the feedback itself is typically fixed by the tool (e.g., compiler error messages, test pass/fail) and treated as a given rather than a design variable. Our work complements this line by holding the verifier constant and systematically varying how its output is *represented* to the LLM.

2.3 Feedback Format and Prompt Representation

While code-feedback studies treat feedback content as fixed by the tool, a parallel line of work has shown that how *any* information is formatted in a prompt can significantly affect model behavior. He et al. [2024] find that GPT-3.5-turbo performance varies by up to 40 percentage points across prompt formats (plain text, Markdown, JSON, YAML), with smaller models exhibiting greater sensitivity. Liu et al. [2025] propose joint content-format optimization, demonstrating that the optimal prompt format is task- and content-dependent. In reasoning, chain-of-thought prompting Wei et al. [2022] and structured decomposition Zhou et al. [2023] show that how intermediate steps are *formatted* can matter as much as their content.

Despite this evidence, no prior work has *disentangled* feedback representation from feedback *source*, studying specificity and format as *independent variables* while holding the underlying verifier constant. The closest work, Dolcetti et al. [2024], varies feedback *source* (tests vs. static analysis) but not its representation. RLTF Liu et al. [2023] varies feedback *granularity* at training time, but this targets reward shaping rather than prompting.

2.4 Positioning of This Work

Our study sits at the intersection of these threads. We design a controlled experiment that first isolates the effect of feedback specificity (Phase A) and then ablates surface format (Phase B), across two

models and two domains, using two deterministic verifiers (geometric property checking for SVG, static analysis for Python security) and two LLMs (Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct). Unlike self-correction studies that conflate feedback generation with feedback utilization, our verifiers are deterministic and domain-specific, isolating the effect of representation. Unlike code-feedback studies that compare feedback-present vs. feedback-absent conditions, we hold feedback content constant across specificity levels and vary only how it is communicated. This design enables the first controlled measurement of feedback representation effects on LLM self-correction across models and domains.

3 Methodology

We study how the *representation* of verifier feedback—its surface format and its level of specificity—affects an LLM's ability to self-correct. Our setup holds the underlying verifier fixed and deterministic, so that any downstream difference in correction quality is attributable to how diagnostics are rendered into natural language or structured text, rather than to what the verifier detects.

3.1 Problem Formulation

Let x denote a task description and y_0 an initial model response (code). A *verifier* V is a deterministic function that maps a candidate y to a multiset of diagnostics $D = V(y) = \{d_1, \ldots, d_k\}$, where each diagnostic d_i identifies a discrete defect (a rule violation, a geometric constraint failure, a security anti-pattern, etc.). A *feedback renderer* $R_{f,s}$ parameterised by a format $f \in \mathcal{F}$ and a specificity level $s \in \mathcal{S}$ projects D into a textual feedback message $R_{f,s}(D) \in \Sigma^*$. Given an LLM π_θ, the *self-correction* step produces a revised response

$$y_1 \sim \pi_\theta(\cdot \mid x,\, y_0,\, R_{f,s}(V(y_0)))\,. \tag{1}$$

The quantity of interest is the reduction in diagnostics $|V(y_0)| \to |V(y_1)|$ as a function of (f, s), holding V, π_θ, x, and the decoding procedure fixed. We ask whether and how feedback *specificity* and *format* interact to affect self-correction quality, and whether these effects generalise across models and task domains.

Unified diagnostic schema. To enable apples-to-apples comparison across heterogeneous verifiers (XML/geometric vs. Python static analysis), we require every diagnostic to conform to a single schema:

Field	Description
`rule_id`	stable identifier of the violated rule
`severity`	{ERROR, WARNING, INFO}
`element_ids`	affected locations (line numbers, SVG ids)
`message_generic`	severity-only description
`message_precise`	detailed description with code context
`metric_name`, `metric_value`	quantitative evidence
`fix_direction`	suggested remediation action
`metadata`	tool-specific payload

This schema decouples the act of *detection* from the act of *presentation*, which is essential for the ablation of Section 3.3.

3.2 Verifier Design

We instantiate two verifiers spanning qualitatively different domains.

SVG geometric verifier. `SVGGeometricVerifier` consumes an SVG string and emits diagnostics along three axes: (i) *XML well-formedness*—parser errors and malformed attributes; (ii) *geometric validity*—coordinate ranges, shape constraints (positive radii, non-degenerate polygons), and viewBox consistency; and (iii) *predicate satisfaction*—task-specific geometric predicates such as "two circles are externally tangent" or "the polygon is convex," supplied with the task. Each violation is recorded as a `Diagnostic` whose `element_ids` point to offending SVG elements, whose `metric_name`

reports the underlying numerical quantity (e.g., centre-to-centre distance minus the sum of radii), and whose `fix_direction` names the predicate to re-satisfy. This verifier is built on top of the SVGenius task suite Cao et al. [2025].

Python static analyser. `PythonStaticVerifier` wraps two mature off-the-shelf tools: Bandit PyCQA [2024a] for security vulnerabilities and Pylint PyCQA [2024b] for code quality. Raw tool outputs are normalised into the unified schema: Bandit severities {HIGH, MEDIUM, LOW} map to {ERROR, WARNING, INFO} respectively, and Pylint categories {*error/fatal*, *warning*, *convention/refactor*} map similarly. Rule ids are prefixed by tool (e.g., `B602_subprocess_popen_with_shell_equals_true`) to preserve provenance. `metric_value` stores Bandit's confidence score or Pylint's severity weighting. Both tools are invoked in their default configurations; we do not fine-tune rule sets.

Both verifiers are fully deterministic given a fixed tool version. They are treated as black boxes by the LLM: the model never sees the verifier's source code, only its rendered output.

3.3 Feedback Representation

The feedback representation is the product of two orthogonal choices.

Format (f). We consider three surface formats for rendering the diagnostic list D:

- RAW_JSON: the list is serialised as a JSON array of objects, one per diagnostic, with field names matching the schema.
- NL: the list is rendered as a bulleted natural-language enumeration, one sentence per diagnostic.
- HYBRID: a natural-language summary followed by a JSON block, intended to combine the readability of NL with the machine-parseable structure of JSON.

In our main experiment, we fix $f = \text{NL}$ and isolate the effect of specificity, leaving the full 3×2 format $\times$ specificity ablation to a follow-up ablation study.

Specificity (s). Each diagnostic is rendered at one of two specificity levels:

- GENERIC: severity tag + `message_generic`. No location, no values, no fix hints.
- PRECISE: `message_precise` + `element_ids` + `metric_value` + `fix_direction`.

Critically, both variants originate from the *same* underlying diagnostic, so differences in downstream self-correction cannot be attributed to detecting different issues.

Example. For a single Bandit finding on a call to `subprocess.call(cmd, shell=True)`, the two renderings are:

GENERIC:
```
[ERROR] subprocess_popen_with_shell_equals_true: Use of subprocess with
shell=True.
```

PRECISE:
```
[ERROR] Line 5: Use of subprocess call with shell=True identified.
Code: `subprocess.call(cmd, shell=True)`. Suggested fix: Fix
B602_subprocess_popen_with_shell_equals_true security issue at line 5.
```

The PRECISE variant reveals *where* the defect lives, *what* the offending expression is, and *how* to begin fixing it—information that the model, by construction, cannot infer from the GENERIC message alone. Figure **??** illustrates the full rendering pipeline.

3.4 Experimental Protocol

Datasets. We use the medium split of SVGenius Cao et al. [2025] for the geometric domain ($n = 100$ tasks) and SecurityEval Siddiq and Santos [2022] for the Python security domain ($n = 121$

Algorithm 1 Single cell evaluation

Require: dataset $\mathcal{D}$, verifier V, model π_θ, renderer $R_{f,s}$
$\mathcal{D}' \leftarrow \{(x, y_0) \in \mathcal{D} : |V(y_0)| > 0\}$ ▷ filter
for each $(x, y_0) \in \mathcal{D}'$ **do**
  $D_{\text{before}} \leftarrow V(y_0)$
  $m \leftarrow R_{f,s}(D_{\text{before}})$ ▷ render feedback
  $p \leftarrow \text{BUILDPROMPT}(x, y_0, m)$
  $y_1 \leftarrow \pi_\theta(p)$ ▷ greedy decode
  $D_{\text{after}} \leftarrow V(y_1)$
  record $(|D_{\text{before}}|, |D_{\text{after}}|, \textit{parse_error}_{\text{after}})$
end for
return per-sample diagnostics counts

code snippets with known security vulnerabilities, which serve as the initial code y_0 to be corrected). After filtering to samples with at least one diagnostic ($b_i > 0$; Section 3.5), the effective sample sizes are $n' = 100$ for SVG (all samples contain at least one geometric-predicate warning) and $n' \approx 49$ for Python. The Python attrition is expected: many SecurityEval vulnerability patterns (e.g., insecure deserialization, hardcoded credentials) fall outside the rule coverage of Bandit and Pylint, so the corresponding samples receive zero diagnostics and are excluded.

Models. We evaluate two open-weights instruction-tuned LLMs of comparable scale: Qwen2.5-7B-Instruct Yang et al. [2024a] and Llama-3.1-8B-Instruct Grattafiori et al. [2024]. Both are served via HuggingFace Transformers with greedy decoding (temperature 0) to make the correction step reproducible.

Grid. Our main experiment sweeps a $2 \times 2 \times 2$ grid of $\{\text{PRECISE}, \text{GENERIC}\} \times \{\text{Qwen2.5-7B}, \text{Llama-3.1-8B}\} \times \{\text{SVG}, \text{Python}\}$, holding the format at NL. Each of the eight cells corresponds to one independent experimental run.

Per-cell procedure. Algorithm 1 summarises the pipeline for a single cell.

The correction prompt p concatenates (i) the task description, (ii) the original code, and (iii) the rendered feedback, under a fixed template shared across all cells. No few-shot exemplars are used.

3.5 Evaluation Metrics

Our primary headline metric is the *macro diagnostic reduction rate*

$$\text{DRR}_{\text{macro}} = \frac{1}{|\mathcal{D}'|} \sum_{i \in \mathcal{D}'} \left(1 - \frac{\widetilde{a}_i}{\max(b_i,\, 1)}\right), \tag{2}$$

where $b_i = |V(y_0^{(i)})|$ is the pre-correction diagnostic count and $\widetilde{a}_i$ is an *effective* post-correction count defined below. A complementary *micro* aggregate pools counts across samples:

$$\text{DRR}_{\text{micro}} = 1 - \frac{\sum_{i \in \mathcal{D}'} \widetilde{a}_i}{\sum_{i \in \mathcal{D}'} b_i}. \tag{3}$$

The macro form weights every sample equally and is used for headline comparisons; the micro form is reported as a robustness check against samples with unusually large b_i.

Parse-error penalty. A naive use of $a_i = |V(y_1^{(i)})|$ can reward the model for *breaking* its output: a response that is no longer parseable may yield only a single *parse_error* diagnostic, superficially lowering the count. We therefore define

$$\widetilde{a}_i = \begin{cases} \max(b_i + 10,\ a_i) & \text{if sample } i \text{ is newly} \\ & \text{a parse_error after correction,} \\ a_i & \text{otherwise.} \end{cases} \tag{4}$$

The penalty $|D_i| + 10$ ensures that newly broken samples always contribute a strongly negative reduction rate, since the effective count exceeds the original by at least 10 diagnostics. The threshold of 10 is a small integer chosen once and applied uniformly; we do not tune it per cell. In a sensitivity check (Appendix B), we verify that conclusions are robust to penalties in the range $[5, 20]$.

Table 1: Phase A results: specificity ablation with NL format. Δ = $\text{DRR}_{\text{macro}}$(precise) − $\text{DRR}_{\text{macro}}$(generic). Sign test and Wilcoxon signed-rank test are paired per-sample comparisons. $^{*}p < 0.05$; $^{**}p < 0.01$; $^{***}p < 0.001$.

Model	Domain	n	$\text{DRR}_{\text{macro}}$(P)	$\text{DRR}_{\text{macro}}$(G)	Δ	Sign p
Qwen	SVG	100	−3.2%	16.7%	−19.9 pp	$<0.001^{***}$
Qwen	Python	49	67.3%	55.1%	+12.2 pp	0.021^{*}
Llama	SVG	100	40.6%	41.4%	−0.8 pp	0.110
Llama	Python	49	73.1%	70.4%	+2.7 pp	0.625

Sample filtering. $\mathcal{D}'$ retains only samples with $b_i > 0$ (any severity), since Eq. 2 is ill-defined when $b_i = 0$ and the correction task has no target.

Secondary metrics. For the SVG domain we additionally report: (i) *parse survival rate*, the fraction of corrected samples that still parse as valid XML, and (ii) *predicate fix rate among survivors*, the fraction of parse-surviving samples whose task-specific geometric predicates are satisfied after correction. For the Python domain we report: (i) *pass rate*, the fraction of samples with $a_i = 0$, and (ii) *error reduction rate*, the same quantity as Eq. 2 restricted to ERROR-severity diagnostics.

Insufficient-samples sentinel. Any cell with $|\mathcal{D}'| < 30$ after filtering is excluded from headline aggregation and flagged as INSUFFICIENT_SAMPLES. This guards against headline claims driven by a handful of samples.

Decision criterion for the headline claim. Let $\Delta_c = \text{DRR}_{\text{macro}}(\text{PRECISE}, c) - \text{DRR}_{\text{macro}}(\text{GENERIC}, c)$ for each of the 4 model×domain cells c. The headline claim—*precise feedback improves diagnostic reduction by at least* 10 *percentage points*—is considered supported if and only if: (a) $\frac{1}{4}\sum_c \Delta_c \geq 0.10$, (b) $|\{c : \Delta_c \geq 0.10\}| \geq 3$, and (c) every cell satisfies $|\mathcal{D}'| \geq 30$. All three conditions must hold jointly; violating any of them downgrades the finding from *headline* to *exploratory* in the results section.

4 Results

4.1 Phase A: The Specificity Effect is Domain-Dependent

Decision gate. Our pre-specified headline claim—that precise feedback improves DRR by at least 10 percentage points across all four model×domain cells—**fails**. The mean Δ across cells is -1.4 pp (far below the 10 pp threshold), and only one of four cells exceeds 10 pp (Table 1). We therefore report Phase A findings as *exploratory*.

Micro DRR results follow the same directional pattern as macro across all eight cells (see Appendix A).

Domain × specificity interaction. Although the overall headline fails, the cell-level pattern reveals a striking interaction: *the direction of the specificity effect reverses between domains*. In the Python security domain, precise feedback consistently improves DRR over generic for both models (Qwen +12.2 pp, Llama +2.7 pp). In the SVG geometric domain, precise feedback is neutral or harmful (Qwen −19.9 pp, Llama −0.8 pp).

For Qwen, the cell-level paired tests support this interaction: the sign test confirms that generic feedback significantly outperforms precise on SVG (46 vs. 9 wins, $p < 0.001$), while precise significantly outperforms generic on Python (9 vs. 1 wins, $p = 0.021$; Wilcoxon $p = 0.041$). The opposing directions of significance across domains suggest a domain×specificity interaction, though we note that these are cell-level tests rather than a formal interaction test. For Llama, the directional pattern is consistent with Qwen but does not reach significance, likely due to Llama's stronger baseline self-correction ability on SVG (40.6% vs. Qwen's −3.2%), which compresses the room for differential effects.

Table 2: Robustness of precise−generic Δ to outlier-resistant estimators. Trim and Winsor at 5%.

Cell	Δ_{mean}	Δ_{trim}	Δ_{winsor}
Qwen SVG	−19.9 pp	−24.0 pp	−24.1 pp
Qwen Python	+12.2 pp	+13.3 pp	+12.2 pp
Llama SVG	−0.8 pp	+7.6 pp	+6.9 pp
Llama Python	+2.7 pp	+3.0 pp	+2.7 pp

Table 3: Phase B results: format × specificity on Python ($n = 49$ per cell). Δ = precise − generic. $^{*}p < 0.05$; $^{**}p < 0.01$. Bonferroni-corrected threshold for 6 tests: $\alpha_{\text{adj}} = 0.0083$.

Model	Format	DRR(P)	DRR(G)	Δ	Sign p	Wilcoxon p
Qwen	NL	67.3%	55.1%	+12.2 pp	0.021^{*}	0.041^{*}
Qwen	raw_json	59.5%	52.0%	+7.5 pp	n.s.	n.s.
Qwen	hybrid	75.5%	56.1%	+19.4 pp	0.003^{**}	0.002^{**}
Llama	NL	73.1%	70.4%	+2.7 pp	n.s.	n.s.
Llama	raw_json	74.2%	79.9%	−5.7 pp	n.s.	n.s.
Llama	hybrid	69.0%	76.2%	−7.2 pp	n.s.	n.s.

Robustness checks. Table 2 reports trimmed and Winsorized mean estimates. The Qwen Python precise advantage is stable across all three estimators (+12.2% to +13.3%), as is the Qwen SVG generic advantage (-19.9% to -24.1%). The Llama SVG Δ is sensitive to extreme values: the raw mean of -0.8 pp shifts to +7.6 pp under 5% trimming, indicating that a small number of outlier samples dominate this cell.

Outlier analysis. The most extreme individual case is Qwen SVG sample #54, which has 1 diagnostic before correction but introduces a parse error after correction (newly broken), inflating its effective post-correction count to 11 via the penalty of Eq. 4. This yields a per-sample reduction of -10.0, heavily dragging Qwen SVG precise's macro mean negative. The sample's output was truncated at the 8192-token limit (token count = 8192), suggesting that the model attempted a verbose SVG rewrite that exceeded the generation budget. Such cases are characteristic of the SVG domain, where geometric code generation is inherently longer than Python snippet correction.

4.2 Phase B: Format Modulates the Specificity Effect

Having established that the specificity effect is domain-dependent, we next ask whether *format* further modulates this effect. Phase B ablates three feedback formats (NL, raw JSON, hybrid) crossed with two specificity levels, on the Python domain where Phase A found the clearest specificity signal.

Qwen: hybrid format amplifies specificity. For Qwen, precise feedback significantly outperforms generic in the hybrid format ($\Delta = +19.4$ pp; sign test $p = 0.003$, Wilcoxon $p = 0.002$), an effect that survives Bonferroni correction for the six pairwise comparisons ($\alpha_{\text{adj}} = 0.0083$). The format gradient under precise feedback is hybrid (75.5%) > NL (67.3%) > raw JSON (59.5%), suggesting that the combination of natural-language readability and structured JSON detail maximises the model's ability to exploit precise diagnostic information.

Llama: structured formats favour generic feedback. Llama exhibits the opposite pattern. Under raw JSON and hybrid formats, generic feedback yields higher DRR than precise (raw JSON: 79.9% vs. 74.2%; hybrid: 76.2% vs. 69.0%), although none of these differences reach significance individually. The best-performing configuration overall is Llama with raw JSON generic feedback (79.9%), which does not significantly benefit from increased specificity. This non-significant trend suggests that Llama may already internalise sufficient corrective knowledge from its training data, such that additional precise details in structured formats introduce noise rather than useful signal.

Format × specificity interaction. The range of the precise−generic Δ across formats is 11.9 pp for Qwen (from +7.5 pp in raw JSON to +19.4 pp in hybrid) and 9.9 pp for Llama (from -7.2 pp in hybrid to +2.7 pp in NL). Bootstrap tests confirm that this range is significantly greater than zero for

Table 4: Top configurations by DRR_{macro} on Python ($n = 49$).

Rank	Model	Format	Specificity	DRR_{macro}
1	Llama	raw_json	generic	79.9%
2	Llama	hybrid	generic	76.2%
3	Qwen	hybrid	precise	75.5%
4	Llama	raw_json	precise	74.2%
5	Llama	NL	precise	73.1%

Table 5: Claim hierarchy with evidence strength.

Finding	Evidence	Language
Qwen hybrid precise is optimal	sign + Wilcoxon $p < 0.01$	"significantly outperforms"
Qwen precise > generic across formats	3/3 positive Δ	"consistently improves"
Llama does not benefit from precise	0/3 significant	"does not significantly benefit"
Llama structured + generic trend	$\Delta < 0$ in 2/3 formats	"non-significant trend"
Model-dependent interaction	opposite optima	"we observe a model-dependent interaction"

both models, indicating that format is not a neutral carrier of feedback content but actively modulates how specificity affects self-correction.

Best configurations. Table 4 ranks the top configurations across both models. The optimal feedback representation differs qualitatively between models: Qwen benefits from maximum information density (hybrid + precise), while Llama performs best with minimal specificity in a structured format (raw JSON + generic).

4.3 Summary of Evidence

Table 5 maps each finding to its evidentiary strength and the corresponding language used throughout this paper.

5 Discussion

5.1 Why Specificity Helps in Python but Not in SVG

The Python security domain is characterised by *rule-based* diagnostics: each finding identifies a specific anti-pattern (e.g., `subprocess` with `shell=True`) at a precise code location, and the corresponding fix is typically a localised substitution. Precise feedback maps directly onto these atomic edits, giving the model an unambiguous corrective signal. Generic feedback, by contrast, names the rule but omits the location and fix direction, forcing the model to re-discover where the problem lies.

In the SVG geometric domain, diagnostics are *structural*: a predicate failure ("circles are not externally tangent") implicates global relationships between multiple elements, and the fix often requires coordinated adjustments to several attributes (centres, radii, viewBox). Precise feedback still localises the failure to specific element IDs and reports metric values, but the model must reason about multi-element geometry to translate this information into a valid edit. For Qwen—whose SVG correction under precise feedback is near zero ($DRR_{macro} = -3.2\%$, compared to 16.7% under generic)—the additional precise details may overwhelm the model's geometric reasoning capacity, producing responses that are longer but no more correct.

5.2 Why Hybrid Amplifies Specificity for Qwen but Not Llama

The hybrid format provides diagnostic information twice: once as a natural-language narrative and once as a structured JSON block. For Qwen, this dual encoding is associated with the strongest effect we observe: hybrid precise achieves 75.5% DRR, 8.2 pp above NL precise and 16.0 pp above raw JSON precise.

The precise mechanism by which hybrid format amplifies specificity for Qwen remains unclear. One hypothesis is that the redundant encoding (NL + JSON) provides two complementary access paths to the same information, which benefits models that struggle with structured input alone. However, we cannot distinguish this from alternative explanations (e.g., increased prompt length per se, or formatting cues that prime different decoding behaviours) without ablating the hybrid components independently. We leave mechanistic investigation to future work.

For Llama, the same redundancy does not help. Llama's higher baseline DRR (NL generic 70.4% vs. Qwen's 55.1%) suggests less room for improvement from external guidance. When presented with hybrid precise feedback—the most information-dense representation—Llama's DRR *drops* to 69.0%, below every other Llama configuration. This is consistent with observations that additional prompt context can degrade performance in some settings Shi et al. [2023], though the mechanism in our case may differ.

5.3 Practical Implications

Our results carry a clear practical message: **there is no one-size-fits-all feedback representation for LLM self-correction**. A system designer who couples a deterministic verifier with an LLM corrector must consider:

1. **Model identity.** Qwen benefits from maximum feedback specificity and the hybrid format, while Llama performs best with generic feedback in a structured format. With only two models of similar scale (7B/8B), we cannot attribute this difference to a single axis such as "capability"; training data composition, instruction-tuning procedure, and architecture may all contribute.
2. **Task domain.** Precise feedback helps for rule-based tasks (Python security) but is neutral or harmful for structural tasks (SVG geometry), at least for models with limited domain-specific reasoning.
3. **Format choice.** Format is not a neutral vehicle—it actively modulates the specificity effect. The interaction is model-dependent, ruling out simple heuristics like "always use the most detailed format."

These findings suggest that feedback pipelines should be *adaptive*: choosing the representation based on the downstream model's characteristics, rather than defaulting to maximum information.

5.4 Limitations

Several limitations constrain the generalisability of our findings.

- **Sample size.** The Python domain uses $n = 49$ samples after filtering, which limits statistical power for individual cell comparisons. The significant effects we report (Qwen hybrid precise) are robust to multiple-comparison correction, but non-significant trends (Llama's preference for generic) may reflect insufficient power rather than true null effects.
- **Model coverage.** We evaluate two 7–8B instruction-tuned models. Whether the observed model-dependent interaction extends to larger models (70B+), different architectures, or proprietary models remains open.
- **Domain coverage.** Phase A covers two domains (SVG, Python), but the format ablation (Phase B) was conducted on the Python domain only. Whether the format $\times$ specificity interaction generalises to SVG or other structural domains remains open.
- **Single correction round.** Our protocol applies one round of correction. Multi-turn feedback may exhibit different dynamics, particularly for the SVG domain where iterative refinement could help models that fail in a single pass.
- **Deterministic verifiers only.** We hold the verifier fixed and deterministic. Stochastic verifiers (e.g., LLM-as-judge) would introduce additional variance and are outside our scope.
- **Greedy decoding.** All experiments use temperature 0 for reproducibility. Stochastic decoding may yield different feedback-representation dynamics.

- **Training data confound.** Qwen and Llama differ in training data composition. The observed model-dependent interaction may partially reflect training-data-specific adaptation rather than model capability per se.

5.5 Future Work

Several directions emerge naturally from our findings. First, scaling the model dimension: testing whether the Qwen-like pattern (precise helps) or the Llama-like pattern (generic suffices) correlates with model scale, instruction-tuning quality, or domain-specific pre-training exposure. Second, extending the format ablation to the SVG domain and other structural tasks (e.g., HTML/CSS layout, circuit design). Third, exploring *adaptive feedback selection*, where the representation is chosen dynamically based on the model's confidence or the verifier's diagnostic pattern. Finally, multi-turn self-correction, where the feedback representation may need to evolve across rounds as the model converges toward a correct solution.

References

Xiaoyu Cao et al. SVGenius: Benchmarking LLMs in SVG understanding, editing and generation. In *ACM MM*, 2025. arXiv:2506.03139.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *ICLR*, 2023.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *ICLR*, 2024.

Greta Dolcetti, Andrea Romanoni, Mattia Poli, Federico Delprino, and Sergio Mezzi. Helping LLMs improve code generation using feedback from testing and static analysis. *arXiv preprint arXiv:2412.14841*, 2024.

Jonas Gehring et al. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *ICML*, 2025.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Jia He et al. Does prompt formatting have any impact on LLM performance? *arXiv preprint arXiv:2411.10541*, 2024.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *ICLR*, 2024.

Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez, and Greg Durrett. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *TACL*, 2024.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, et al. Competition-level code generation with AlphaCode. *Science*, 2022.

Jiate Liu et al. RLTF: Reinforcement learning from unit test feedback. *TMLR*, 2023.

Yuanye Liu, Jiahang Xu, et al. Beyond prompt content: Enhancing LLM performance via content-format integrated prompt optimization. *arXiv preprint arXiv:2502.04295*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *ICML*, 2023.

Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *ICLR*, 2024.

PyCQA. Bandit: A tool designed to find common security issues in Python code. `https://github.com/PyCQA/bandit`, 2024a.

PyCQA. Pylint: Static code analyser for Python. `https://github.com/pylint-dev/pylint`, 2024b.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *ICML*, 2023.

Yuling Shi et al. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint arXiv:2410.01215*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

Mohammed Latif Siddiq and Joanna C.S. Santos. SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques. In *MSR4PS Workshop*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

An Yang et al. Qwen2.5: A party of foundation models. *arXiv preprint arXiv:2412.15115*, 2024a.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024b.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *NeurIPS*, 2024c.

Li Zhong and Zilong Wang. LDB: A large language model debugger via verifying runtime execution step by step. In *ACL Findings*, 2024.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *ICLR*, 2023.

A Micro DRR Results

Tables 6 and 7 report micro DRR (Eq. 3), which pools diagnostic counts across samples before computing the reduction rate. Micro DRR down-weights samples with few diagnostics and up-weights samples with many, complementing the macro (per-sample mean) metric used in the main text.

Table 6: Phase A micro DRR results. $\Delta_{\text{micro}} = \text{DRR}_{\text{micro}}(\text{precise}) - \text{DRR}_{\text{micro}}(\text{generic})$.

Model	Domain	n	$\text{DRR}_{\text{micro}}$(P)	$\text{DRR}_{\text{micro}}$(G)	Δ_{micro}
Qwen	SVG	100	2.7%	16.0%	−13.3 pp
Qwen	Python	49	67.2%	53.7%	+13.5 pp
Llama	SVG	100	30.3%	36.8%	−6.5 pp
Llama	Python	49	71.6%	68.7%	+2.9 pp

All four cells show the same directional pattern as macro DRR (Table 1): precise feedback improves DRR on Python but reduces it on SVG for both models. The magnitude of the Qwen SVG penalty is smaller under micro (−13.3 pp vs. −19.9 pp macro), consistent with the outlier analysis in Section 4.1: the macro metric amplifies extreme per-sample values (e.g., sample #54), while micro aggregation dilutes them.

Phase B micro results mirror the macro pattern (Table 3): Qwen benefits from precise feedback across all three formats (hybrid > NL > raw_json), while Llama shows a reversed pattern under structured formats (raw_json and hybrid favour generic). The model-dependent interaction persists under both aggregation methods.

Table 7: Phase B micro DRR results on Python ($n = 49$ per cell). Δ_{micro} = precise − generic.

Model	Format	$\text{DRR}_{\text{micro}}$(P)	$\text{DRR}_{\text{micro}}$(G)	Δ_{micro}
Qwen	NL	67.2%	53.7%	+13.5 pp
Qwen	raw_json	58.2%	50.7%	+7.5 pp
Qwen	hybrid	74.6%	55.2%	+19.4 pp
Llama	NL	71.6%	68.7%	+2.9 pp
Llama	raw_json	73.1%	77.6%	−4.5 pp
Llama	hybrid	68.7%	74.6%	−5.9 pp

B Sensitivity Analysis

The parse-error penalty (Eq. 4) adds a fixed offset of $+10$ to the effective diagnostic count for newly broken samples. Table 8 reports Qwen SVG precise $\text{DRR}_{\text{macro}}$—the cell most affected by the penalty due to outlier sample #54—under alternative penalty values.

Table 8: Sensitivity of Qwen SVG precise $\text{DRR}_{\text{macro}}$ to the parse-error penalty threshold. The cell contains $n = 100$ samples, of which 3 are newly broken after correction.

Penalty	$+5$	$+8$	$+10$ (default)	$+15$	$+20$
$\text{DRR}_{\text{macro}}$	-1.7%	-2.7%	-3.2%	-4.7%	-6.2%
Δ (P−G)	−18.4 pp	−19.4 pp	−19.9 pp	−21.4 pp	−22.9 pp

Across the $[5, 20]$ range, Qwen SVG precise $\text{DRR}_{\text{macro}}$ remains negative and the precise−generic gap stays large (−18.4 to −22.9 pp). The qualitative conclusion—that generic feedback substantially outperforms precise on Qwen SVG—is robust to the choice of penalty. For all other cells, the number of newly broken samples is ≤ 1, making the penalty value inconsequential.